DEEP EXPLANATION NOTES | DIAGRAMS | LIVE EXAMPLES

Java Web Technology

Servlet | JSP | MVC | EJB | Session

- MVC + Architecture
- JavaBean Theory
- EJB & Session Beans
- Servlet vs JSP Session
- Servlet & JSP Life Cycle
- 9 Implicit Objects
- JSP Elements / Scriptlet
- Interview Q&A

Start: Open Index >

Tip: Index ke cards click karo. Har page pe "Back to Index" link hai.
PDF reader ke Bookmarks panel se bhi chapters pe jump kar sakte ho.

INDEX

Kisi bhi **card pe tap / click** karo - seedha us chapter pe pahunch jaoge. Har page ke neeche **Back to Index** link hai, aur PDF reader ke left **Bookmarks** panel se bhi jump kar sakte ho.

1 **MVC Pattern + Live Example**
Model-View-Controller, flow, Student Result app

2 **Architecture (Java EE / Web)**
Tiers, web container, online banking example

3 **Beans Theory (JavaBean)**
Rules, properties, useBean, JavaBean vs EJB

4 **EJB - Enterprise JavaBeans**
Container, types, services, code

5 **Session & Session Tracking**
HTTP stateless, cookies, HttpSession API

6 **Session Beans (EJB)**
Stateless, Stateful, Singleton + lifecycle

7 **Servlet Session vs JSP Session**
Kya same hai? Full difference table

8 **JSP vs Servlet**
13-point difference + code comparison

9 **Servlet & JSP Life Cycle**
init/service/destroy, jspInit/_jspService

10 **What is JSP & Why JSP**
Definition, need, advantages, anatomy

11 **Which is Faster: JSP or Servlet?**
Sach, first-request cost, tuning tips

12 **9 Implicit Objects of JSP**
request, response, out ... exception

13 **JSP Elements & Scriptlet**
Scriptlet, expression, declaration, directive

14 **Revision, Q&A, Cheat Sheet**
Interview questions + mnemonics

Is PDF me kaise padhein? (Colour guide)

NOTE
Blue box = extra jaankari / yaad rakhne wali baat.

TIP
Green box = best practice / shortcut.

WARNING
Orange box = common galti ya danger zone.

EXAM / INTERVIEW POINT
Purple box = exam ya interview me seedha poochi jaane wali line.

Note: Code me `javax.servlet.*` use kiya hai (syllabus/older Tomcat 9 style). Tomcat 10+ / Jakarta EE 9+ me package `jakarta.servlet.*` ho gaya hai - sirf package ka naam badla hai, concept wahi hai.

1 MVC Pattern + Live Example

Model - View - Controller

1.1 MVC kya hai?

MVC (Model-View-Controller) ek **design pattern** hai jo application ko 3 alag hisson me baant deta hai taaki har hisse ka kaam clear rahe aur ek hissa badalne par baaki na tootein. Isi idea ko **Separation of Concerns** kehte hain.

MODEL (data + logic)	VIEW (UI)	CONTROLLER (manager)
Kaam: DB se data lana/save karna, business rules (marks se grade nikalna). **Java Web me:** JavaBean, POJO, DAO, Service, EJB. **Ko pata nahi:** data screen pe kaise dikhega.	**Kaam:** Model ka data user ko dikhana. **Java Web me:** JSP, HTML, EL, JSTL. **Rule:** isme DB code / heavy logic nahi.	**Kaam:** request lena, input check karna, Model ko call karna, sahi View chunna. **Java Web me:** Servlet (ya Filter). **Rule:** isme HTML likhna nahi.

> **NOTE**
> **Restaurant analogy:** Customer = Browser, Waiter = Controller (order leta hai, kitchen ko bolta hai), Kitchen = Model (asli khana banata hai), Plate presentation = View (khana kaise serve hoga). Waiter khana nahi banata aur kitchen customer se baat nahi karta.

1.2 MVC Architecture Diagram

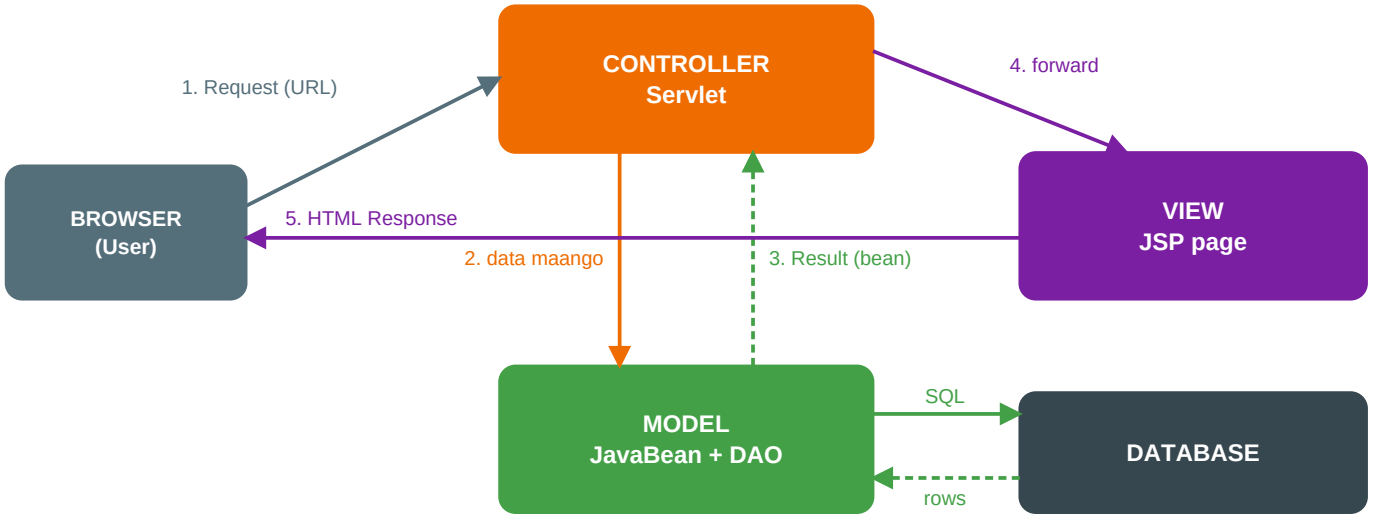


Fig 1.1 - Java Web me MVC (Model 2) ka request flow

Flow step by step

- **1.** User browser me URL/form submit karta hai -> request **Controller (Servlet)** tak aati hai.
- **2.** Controller parameters padhta hai (`request.getParameter`) aur Model (DAO/Bean) ko kaam deta hai.
- **3.** Model DB se baat karke data ek **bean** me bharta hai aur Controller ko wapas deta hai.
- **4.** Controller bean ko `request.setAttribute()` se request me rakhta hai aur **View (JSP)** ko `forward()` karta hai.
- **5.** JSP bean ka data HTML me daalkar **response** banati hai.
- **6.** HTML browser me user ko dikhta hai.

1.3 Live Example: Student Result Portal

Scenario: Student apna roll number daalta hai aur result (naam, marks, grade) dekhta hai. Neeche har file ek MVC part hai.

Project

```
StudentApp/
 |-- src/com/app/model/Student.java            -> MODEL  (JavaBean, data holder + grade rule)
 |-- src/com/app/dao/StudentDAO.java           -> MODEL  (JDBC se DB access)
 |-- src/com/app/controller/ResultServlet.java -> CONTROLLER (request handle, forward)
 |-- WebContent/index.html                     -> input form
 |-- WebContent/result.jsp                     -> VIEW   (sirf display)
```

Step 1: index.html (input form)

HTML

```
<form action="result" method="post">
   Roll No: <input type="text" name="roll">
   <input type="submit" value="Result Dekho">
</form>
```

Step 2: Student.java (Model - JavaBean)

Java

```
package com.app.model;
import java.io.Serializable;

public class Student implements Serializable {
    private int roll;
    private String name;
    private int marks;

    public Student() { }                                    // no-arg constructor (bean rule)
    public int getRoll()           { return roll; }
    public void setRoll(int roll)  { this.roll = roll; }
    public String getName()        { return name; }
    public void setName(String n)  { this.name = n; }
    public int getMarks()          { return marks; }
    public void setMarks(int m)    { this.marks = m; }
    public String getGrade()       { return marks >= 75 ? "A" : marks >= 50 ? "B" : "C"; }  // business rule
}
```

Step 3: StudentDAO.java (Model - DB logic)

Java

```
public class StudentDAO {
    public Student findByRoll(int roll) throws Exception {
        String sql = "SELECT name, marks FROM student WHERE roll = ?";
        try (Connection con = DBUtil.getConnection();
             PreparedStatement ps = con.prepareStatement(sql)) {
            ps.setInt(1, roll);
            ResultSet rs = ps.executeQuery();
            if (rs.next()) {
                Student s = new Student();
                s.setRoll(roll);
                s.setName(rs.getString("name"));
                s.setMarks(rs.getInt("marks"));
                return s;                          // bean bharke wapas
            }
        }
        return null;                               // roll nahi mila
    }
}
```

Step 4: ResultServlet.java (Controller)

Java

```
@WebServlet("/result")
public class ResultServlet extends HttpServlet {
    protected void doPost(HttpServletRequest req, HttpServletResponse res)
            throws ServletException, IOException {
        int roll = Integer.parseInt(req.getParameter("roll"));     // 1. input lo
        try {
            Student s = new StudentDAO().findByRoll(roll);          // 2. Model se data lo
            req.setAttribute("student", s);                         // 3. request me rakho
            req.getRequestDispatcher("result.jsp").forward(req, res); // 4. View ko do
        } catch (Exception e) {
            throw new ServletException(e);
        }
    }
}
```

Step 5: result.jsp (View - sirf display)

JSP

```
<%@ page contentType="text/html;charset=UTF-8" %>
<%@ taglib uri="http://java.sun.com/jsp/jstl/core" prefix="c" %>
<html><body>
  <c:if test="${empty student}"> <h3>Roll number nahi mila!</h3> </c:if>
  <c:if test="${not empty student}">
     <h2>${student.name}</h2>
     Marks: ${student.marks} | Grade: ${student.grade}
  </c:if>
</body></html>
```

EXAM / INTERVIEW POINT

Golden rule: JSP me business logic / DB code mat likho, Servlet me HTML mat likho. Yehi MVC ka poora matlab hai. Controller "kya karna hai" decide karta hai, View "kaisa dikhna hai", Model "data aur rules".

1.4 Model 1 vs Model 2

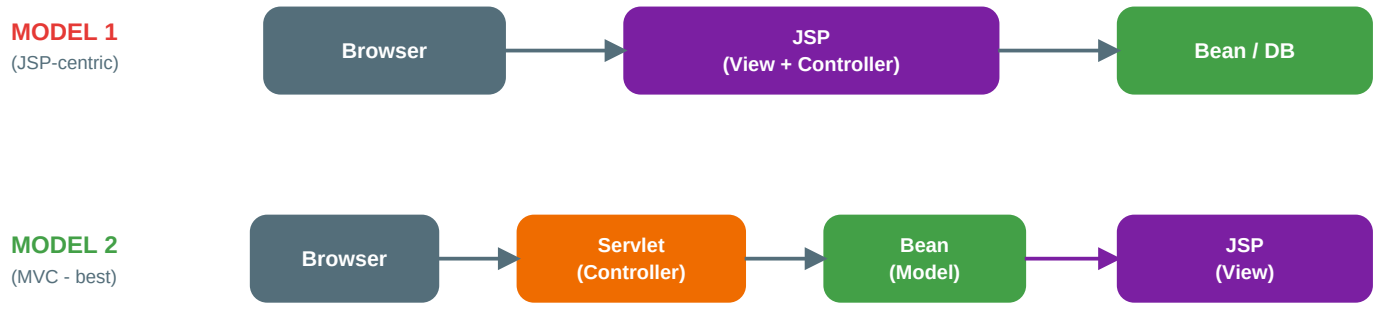


Fig 1.2 - Model 1 (JSP hi sab sambhalta hai) vs Model 2 (MVC)

Point	Model 1	Model 2 (MVC)
Controller	JSP khud	Servlet
Complexity	Chhoti app ke liye easy	Thoda setup, par scalable
Maintenance	Logic + HTML mixed, mushkil	Alag-alag, easy
Reuse / Testing	Kam	Zyada (bean/DAO alag test ho sakte hain)
Use	Demo / small page	Real projects, Struts, Spring MVC

MVC ke fayde aur nuksaan

- **Fayde:** parallel development (designer JSP, developer Java), code reuse, easy testing, ek model ke kai views (web, mobile).
- **Nuksaan:** chhoti app ke liye extra files/complexity, flow samajhne me shuru me time lagta hai.
- **Frameworks:** Struts, Spring MVC, JSF sab MVC hi implement karte hain (Spring me controller = DispatcherServlet + @Controller).

2 Architecture (Java EE / Web)

Tiers, containers aur ek live banking example

2.1 Architecture ka matlab

Application architecture batata hai ki application ke **parts (UI, logic, data)** kahan chalte hain aur aapas me kaise baat karte hain. Alag-alag **tiers** (layers jo alag machine/process par ho sakte hain) se application scalable aur secure banti hai.

Type	Kaisa hota hai	Example	Problem
1-Tier	UI + logic + data ek hi jagah	MS Access / desktop app	Share/scale nahi hota
2-Tier (Client-Server)	Client (UI+logic) <-> DB server	Swing app + JDBC directly	Logic har client me, security kamzor
3-Tier	Presentation <-> Business logic <-> Data	Browser + Servlet/JSP + DB	Best balance
n-Tier (Java EE)	3-tier + web tier / business tier / EIS alag	Browser + Tomcat + EJB server + DB	Complex par bahut scalable

2.2 Java EE Multi-Tier Architecture

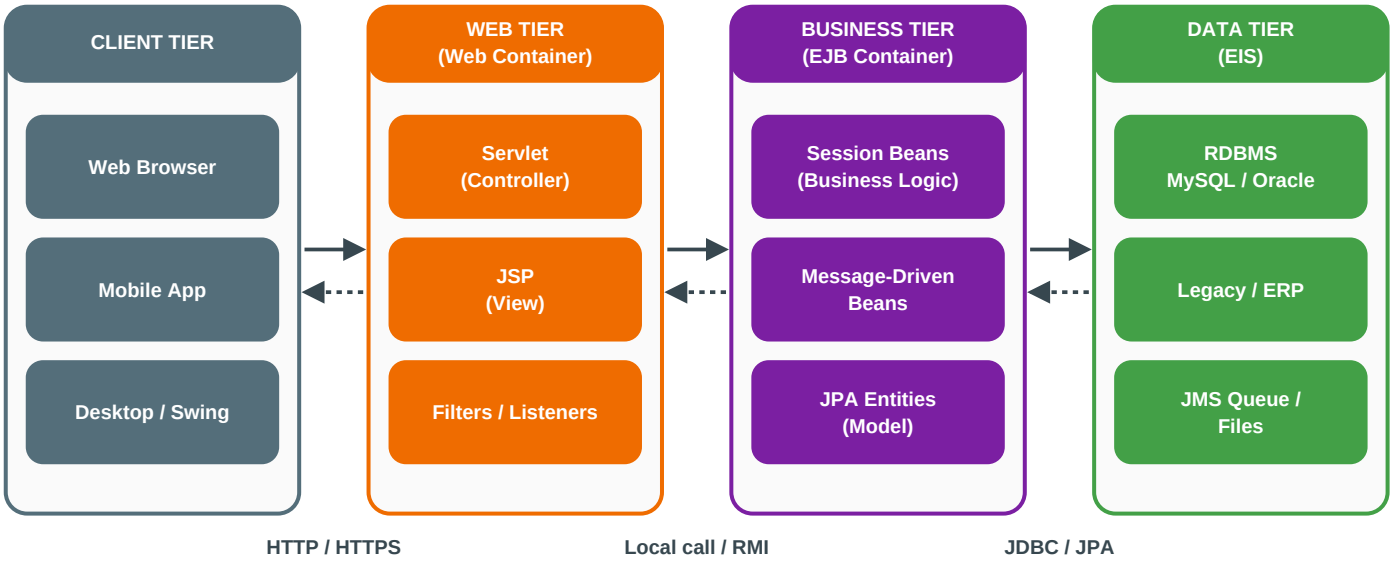


Fig 2.1 - Java EE (Jakarta EE) ka n-tier architecture

Client Tier	Web Tier	Business Tier	Data (EIS) Tier
Browser / mobile jo HTTP request bhejta hai aur HTML/JSON dekhta hai.	**Web Container** (Tomcat) me Servlet, JSP, Filter, Listener. Request handling aur presentation.	**EJB Container** me Session Beans, MDB, JPA. Asli business rules, transactions.	Database, legacy systems, message queues. Sirf data store karta hai.

2.3 Container kya hota hai?

Container ek runtime environment hai jo components (Servlet, JSP, EJB) ki **lifecycle, threading, security, transaction** sambhalta hai. Hum sirf business code likhte hain, baaki plumbing container karta hai.

- **Web Container (Servlet Container):** Servlet + JSP chalata hai. Example: **Apache Tomcat**, Jetty.
- **EJB Container:** Enterprise Beans chalata hai (transaction, pooling, security).
- **Application Server:** Web + EJB + JMS + JPA sab ek saath. Example: **WildFly, GlassFish/Payara, WebLogic, WebSphere**.

EXAM / INTERVIEW POINT

Tomcat sirf web container hai (Servlet/JSP). Plain Tomcat me EJB nahi chalta - uske liye TomEE ya full application server chahiye.

2.4 Live Example: Online Bank - Fund Transfer

User Rs. 5000 apne dost ko transfer karta hai. Dekho har tier kya karta hai:

Step	Tier	Kya hota hai
1	Client	User form bharta hai, `POST /transfer` request HTTP par jaati hai.
2	Web Container	Request **TransferServlet** (Controller) tak pahunchti hai. Wo amount, account validate karta hai.
3	Web -> Business	Servlet `@EJB TransferBean` ka method `transfer(from,to,amt)` call karta hai.
4	Business (EJB)	Bean transaction start karta hai: debit + credit. Dono ho ya dono fail (atomic). Container transaction manage karta hai.
5	Data Tier	JPA/JDBC se **accounts** table update hoti hai; commit.
6	Web Tier	Servlet result bean ko request me rakhkar **success.jsp** ko forward karta hai.
7	Client	Browser me "Transfer Successful" page dikhta hai.

TIP

Is architecture ka fayda: Web tier badalna ho (JSP -> React) to business tier ko haath nahi lagana. DB badalna ho to sirf data tier badlega. Load badhe to web aur business servers alag-alag scale karo.

3 Beans Theory (JavaBean)

Rules, properties, JSP bean tags

3.1 Bean kya hai?

JavaBean ek simple **reusable Java class** hai jo kuch fixed conventions follow karti hai taaki tools, JSP aur frameworks usse automatically samajh sakein. Bean ka kaam mainly **data hold karna** aur use ek jagah se dusri jagah le jaana hota hai (Servlet -> JSP).

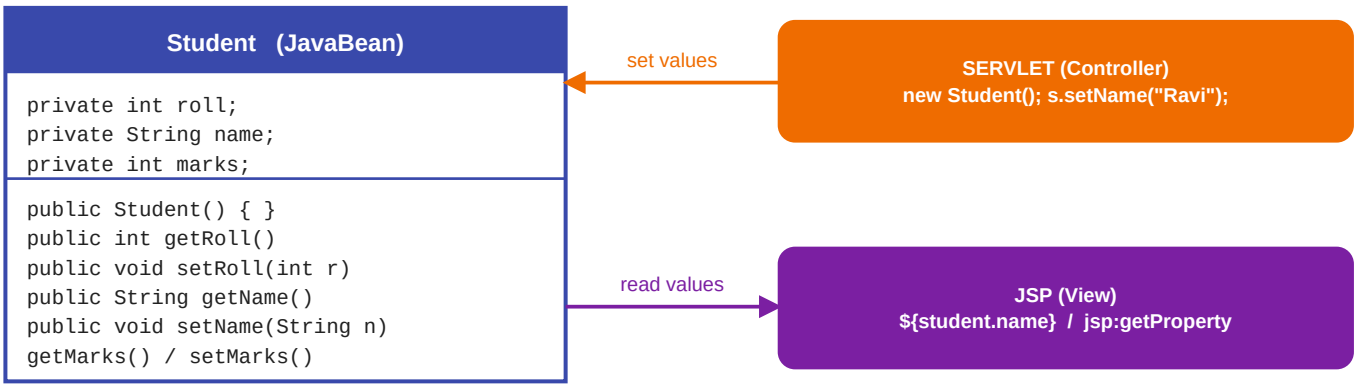


Fig 3.1 - JavaBean ka structure aur Servlet/JSP se uska use

3.2 JavaBean ke 5 rules

#	Rule	Kyon?
1	**public class**	Container/JSP kahin se bhi access kar sake.
2	**public no-argument constructor**	`jsp:useBean` / reflection bina argument ke object banata hai.
3	**private properties (fields)**	Encapsulation: data seedha nahi chhuaya jata.
4	**public getter/setter**: `getXxx()` / `setXxx()` (boolean ke liye `isXxx()`)	Naming convention se property ka naam pata chalta hai: `getName()` -> property **name**.
5	**Serializable** implement kare (recommended)	Session me store karne/disk par save karne ke liye.

> **WARNING**
> Property ka naam getter se banta hai, field se nahi. `getFirstName()` -> property **firstName**. EL me `${emp.firstName}` likhoge. Naming galat to PropertyNotFoundException aata hai.

3.3 Bean ki properties ke types

Type	Matlab	Example
Simple	Ek value	`String name`
Indexed	Array/list jaisi multiple values	`String[] hobbies`, `getHobbies(int i)`
Bound	Value badalne par listener ko notification (PropertyChangeListener)	GUI beans
Constrained	Listener value badalne se **veto** kar sakta hai	Validation in GUI beans

3.4 JSP me Bean use karna (Action tags)

JSP

```
<%-- 1. bean banao ya existing use karo --%>
<jsp:useBean id="emp" class="com.app.model.Employee" scope="session" />

<%-- 2. form ke saare fields ko bean me bharo (naam match hone chahiye) --%>
<jsp:setProperty name="emp" property="*" />

<%-- ya ek-ek property --%>
<jsp:setProperty name="emp" property="name" value="Amit" />

<%-- 3. value padho --%>
Naam: <jsp:getProperty name="emp" property="name" />
Ya EL se: ${emp.name}
```

useBean ka kaam: pehle `scope` me id naam ka object dhundhta hai. **Mil gaya** to wahi reuse karta hai, **nahi mila** to naya object banakar us scope me rakh deta hai.

scope	Bean kitni der tak jeevit	Kahan store hota hai
page (default)	Sirf usi JSP page tak	PageContext
request	Ek request (forward/include samet)	HttpServletRequest
session	Ek user ke poore session tak	HttpSession
application	Poori web-app, sab users	ServletContext

3.5 "Bean" shabd ke alag-alag matlab

Term	Kya hai	Kahan chalta hai	Kis liye
POJO	Koi bhi plain Java class, koi rule nahi	Kahin bhi	General objects
JavaBean	POJO + bean rules (no-arg, getters/setters)	JVM / Web container	Data transfer, JSP, GUI
Enterprise Bean (EJB)	Server-side business component	**EJB container**	Business logic, transactions
Spring Bean	Spring IoC container ka managed object	Spring container	Dependency injection
Managed / CDI Bean	JSF/CDI container se managed	Jakarta EE	JSF backing bean

EXAM / INTERVIEW POINT
JavaBean vs EJB: JavaBean = chhoti reusable class (data/UI), container ki zaroorat nahi. EJB = heavy-weight server component jo EJB container me chalta hai aur transaction/security/remote access deta hai. Dono ka naam "Bean" hai par purpose alag.

4 EJB - Enterprise JavaBeans

Server-side business components

4.1 EJB kya hai?

EJB (Enterprise JavaBeans) Java EE ka **server-side component model** hai jisme hum **business logic** likhte hain. EJB **EJB container** ke andar chalta hai, aur container hume **transaction, security, pooling, concurrency, remote access** jaise heavy kaam automatically deta hai.

4.2 EJB kyon? (Bina EJB ke problem)

- Har project me transaction commit/rollback, security check, connection pooling manually likhna padta.
- Business logic servlet me bharne se code mix ho jata hai aur reuse nahi hota.
- Remote machine se method call (distributed app) manually socket/RMI se banana padta.
- EJB me sirf **annotation** lagao (`@Stateless`), baaki container sambhal leta hai.

4.3 EJB Architecture

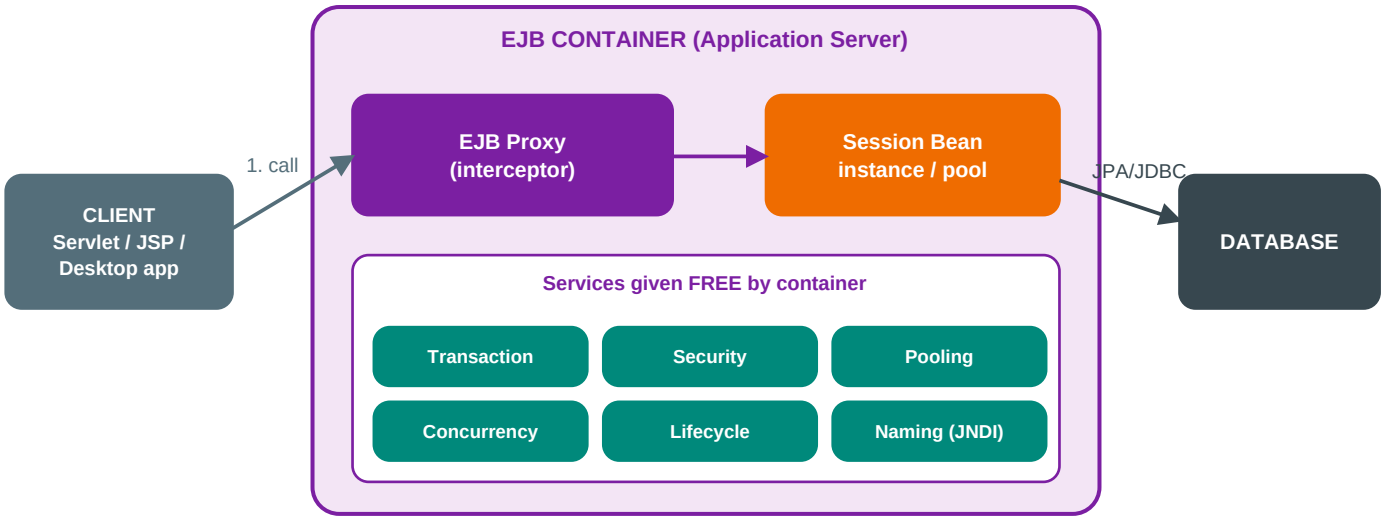


Fig 4.1 - Client kabhi bean se seedha baat nahi karta, container ka proxy beech me hota hai

Client EJB ko call karta hai to pehle **proxy/stub** milta hai. Container is proxy ke through **interceptor** chalata hai: transaction shuru, security check, instance pool se bean nikalna, method run, commit, bean wapas pool me.

4.4 EJB ke types

Type	Kaam	Kab use karein
Session Bean (Stateless / Stateful / Singleton)	Client ke liye business operation karta hai (synchronous)	Transfer, order place, interest calculate
Message-Driven Bean (MDB)	JMS queue/topic ka message **asynchronously** sunta hai. Client ko bean ka reference nahi milta	Email bhejna, order background me process karna
Entity Bean (old, EJB 2.x)	DB row ko represent karta tha	Ab **deprecated**; uski jagah **JPA @Entity**

4.5 Container ki free services

- **Transaction management** - `@TransactionAttribute`, automatic commit/rollback.
- **Security** - `@RolesAllowed("ADMIN")`.
- **Instance pooling** - bean objects reuse, baar-baar create nahi.
- **Concurrency** - thread-safety container sambhalta hai (stateless/stateful me).

- **Dependency injection** - `@EJB`, `@PersistenceContext`.
- **Remote access** - dusri JVM/machine se call (RMI/IIOP).
- **Timer service, interceptors, lifecycle callbacks**.

Local vs Remote interface

	@Local (or no-interface)	@Remote
Client	Same JVM / same application (Servlet -> EJB)	Dusri JVM ya dusra server
Speed	Fast (direct reference, pass-by-reference)	Slower (network, pass-by-value, serialization)
Use	Zyadatar web apps	Distributed systems / remote clients

4.6 Code Example: Stateless EJB + Servlet

Java

```
// ---- Business logic (EJB) ----
@Stateless
public class InterestBean {
    public double simpleInterest(double p, double r, double t) {
        return (p * r * t) / 100;
    }
}
```

Java

```
// ---- Controller (Servlet) ----
@WebServlet("/si")
public class SIServlet extends HttpServlet {
    @EJB                                    // container bean inject karta hai (new nahi karna!)
    private InterestBean bean;

    protected void doGet(HttpServletRequest req, HttpServletResponse res) throws IOException {
        double si = bean.simpleInterest(10000, 8, 2);
        res.getWriter().println("Simple Interest = " + si);     // 1600.0
    }
}
```

Java

```
// ---- Message-Driven Bean (asynchronous) ----
@MessageDriven(mappedName = "jms/OrderQueue")
public class OrderMDB implements MessageListener {
    public void onMessage(Message m) {
        // queue me message aate hi container ye method chala deta hai
    }
}
```

WARNING

EJB object kabhi `new InterestBean()` se mat banao. Phir container ki services (transaction, security) nahi milengi. Hamesha `@EJB` / JNDI lookup.

EXAM / INTERVIEW POINT

EJB ke fayde: rapid development, scalability, built-in transaction/security, portability. **Nuksaan:** heavy, application server chahiye, chhoti apps ke liye overkill (isiliye Spring popular hua; modern EJB 3.x/4.0 lightweight hai).

5 Session & Session Tracking

HTTP stateless hai, to user ko yaad kaise rakhein?

5.1 Problem: HTTP stateless hai

HTTP protocol **stateless** hai - har request **naya, independent** hoti hai, server ko pichli request yaad nahi rehti. Isse login, shopping cart, preferences kaam nahi karenge. Iska solution hai **Session Tracking**.

> **NOTE**
> **Session** = ek hi user ki browser se ki gayi **multiple related requests ki series** (login se logout/timeout tak). Server har user ke liye ek alag **session object** memory me rakhta hai jisme us user ka data hota hai.

5.2 Session Tracking ke 4 tareeke

Technique	Kaise kaam karta hai	Fayda	Nuksaan
Cookies	Server `Set-Cookie` bhejta hai, browser har request me wapas bhejta hai	Simple, automatic	User disable kar sakta hai, size limit (~4KB)
URL Rewriting	URL ke end me ID: `page;jsessionid=ABC123` (`response.encodeURL()`)	Cookie disabled me bhi chalta	URL lamba, ID leak ka risk
Hidden Form Fields	`<input type="hidden" ...>` me data	Cookies nahi chahiye	Sirf form submit par; page source me dikhta hai
HttpSession	Server-side object; ID cookie ya URL se jaati hai	Secure, bada data, easiest	Server memory use karta hai

5.3 HttpSession kaise kaam karta hai

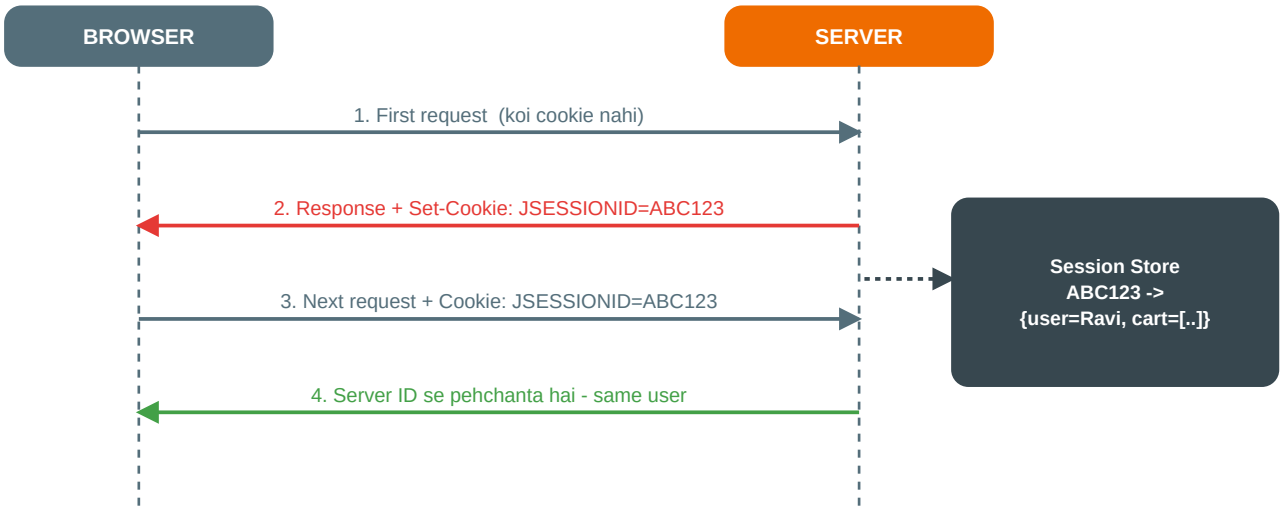


Fig 5.1 - JSESSIONID cookie ke through server user ko pehchanta hai

5.4 HttpSession ke important methods

Method	Kaam
`request.getSession()`	Session lao; **nahi hai to naya banao**
`request.getSession(false)`	Sirf existing lao; **nahi hai to null** (login check ke liye best)
`setAttribute(name, obj)` / `getAttribute(name)`	Data store / retrieve (return type Object - cast karo)
`removeAttribute(name)`	Ek attribute hatao
`invalidate()`	Poora session destroy (logout)
`setMaxInactiveInterval(sec)`	Idle timeout seconds me (-1 = kabhi nahi)
`getId()` / `isNew()`	Session ID / naya hai ya nahi
`getCreationTime()` / `getLastAccessedTime()`	Time info (milliseconds)

5.5 Live Example: Login -> Dashboard -> Logout

Java

```
// LoginServlet
protected void doPost(HttpServletRequest req, HttpServletResponse res) throws IOException {
    String u = req.getParameter("user"), p = req.getParameter("pass");
    if (new UserDAO().isValid(u, p)) {
        req.changeSessionId();                          // session fixation se bachao
        HttpSession session = req.getSession();         // naya session
        session.setAttribute("user", u);                // data store
        session.setMaxInactiveInterval(15 * 60);        // 15 min idle timeout
        res.sendRedirect("dashboard");
    } else {
        res.sendRedirect("login.html?error=1");
    }
}
```

Java

```
// DashboardServlet - protected page
protected void doGet(HttpServletRequest req, HttpServletResponse res) throws IOException {
    HttpSession session = req.getSession(false);       // naya session mat banao
    if (session == null || session.getAttribute("user") == null) {
        res.sendRedirect("login.html");                // login nahi hai
        return;
    }
    String user = (String) session.getAttribute("user");
    res.getWriter().println("Welcome " + user);
}

// LogoutServlet
HttpSession s = req.getSession(false);
if (s != null) s.invalidate();                         // session khatam
res.sendRedirect("login.html");
```

XML

```
<!-- web.xml : timeout (minutes) -->
<session-config>
    <session-timeout>30</session-timeout>
</session-config>
```

TIP

Security tips: login ke baad naya session ID do (`changeSessionId()`), cookie ko **HttpOnly + Secure** rakho, session me bahut bada object mat rakho, aur sensitive data (password) session me store mat karo.

6 Session Beans (EJB)

Stateless | Stateful | Singleton

6.1 Session Bean kya hai?

Session Bean ek EJB hai jo **ek client ki taraf se business task** perform karta hai (jaise paisa transfer, cart manage). Ise "session" isliye kehte hain kyunki ye client ke **ek interaction/session** ko represent karta hai.

> **WARNING**
> **Confusion mat karo: HttpSession** (Servlet API, web tier, user ka data) aur **Session Bean** (EJB, business tier, business logic) **alag cheezein** hain. Dono ka naam "session" hai bas.

6.2 Teen types

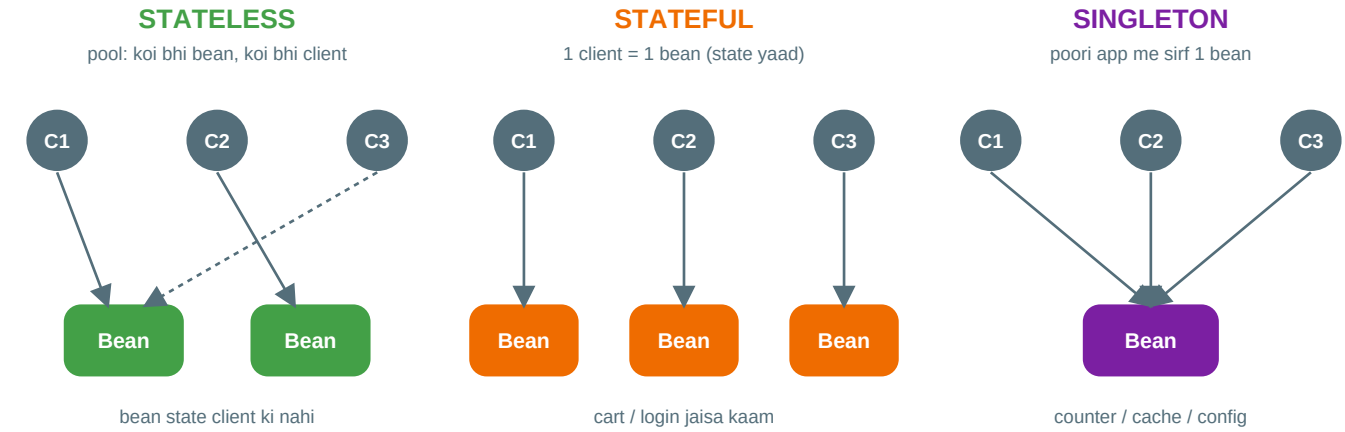


Fig 6.1 - Client aur bean instance ka relation (C = client)

Point	Stateless	Stateful	Singleton
Annotation	`@Stateless`	`@Stateful`	`@Singleton`
State	Client ki conversation yaad nahi rakhta	Client-specific state yaad rakhta hai (instance variables)	Poori app ke liye shared state
Instances	Pool me kai; har call pe koi bhi	1 client = 1 instance	Container me sirf **1**
Concurrency	Container ek time pe ek bean pe ek thread deta hai	Same	Container-managed (`@Lock` READ/WRITE) ya bean-managed
Performance	Sabse fast aur scalable	Memory zyada (har client ka bean)	Fast, par lock contention
Passivation	Nahi	**Haan** (idle bean disk pe)	Nahi
Use case	Calculator, payment, email service	Shopping cart, multi-step wizard	Counter, cache, app config
Web ka similar	(utility service)	HttpSession jaisa	ServletContext jaisa

6.3 Stateful bean ki Lifecycle



Fig 6.2 - Stateful session bean life cycle

Callback	Kab chalta hai
`@PostConstruct`	Bean bana + injection hua, uske turant baad (init ka kaam)
`@PrePassivate`	Idle bean ko memory se hata kar disk pe likhne se pehle (resources close karo)
`@PostActivate`	Disk se wapas aane ke baad (resources dobara kholo)
`@Remove`	Is method ke call par container bean ko khatam karega (checkout/logout)
`@PreDestroy`	Bean destroy hone se just pehle (cleanup)

NOTE

Stateless/Singleton lifecycle: Does Not Exist -> Ready (@PostConstruct) -> Does Not Exist (@PreDestroy). Passive state sirf **stateful** me hoti hai.

6.4 Code Examples

Java

```
@Stateful
public class CartBean implements Serializable {
    private List<String> items = new ArrayList<>();       // client-specific state

    public void add(String item)     { items.add(item); }
    public List<String> getItems()   { return items; }

    @Remove
    public void checkout() { /* order DB me save */ items.clear(); }

    @PostConstruct void init()    { System.out.println("Cart ready"); }
    @PreDestroy   void cleanup()  { System.out.println("Cart destroyed"); }
}
```

Java

```
@Singleton
@Startup                                          // app start hote hi bean ban jaye
public class VisitorCounterBean {
    private int count;

    @Lock(LockType.WRITE)  public int hit() { return ++count; }   // ek time pe ek thread
    @Lock(LockType.READ)   public int get() { return count; }     // kai readers ek saath
}
```

EXAM / INTERVIEW POINT

Kaunsa kab? State yaad rakhni hai (cart) -> **Stateful**. Sirf kaam karna hai, state nahi (calculation, payment) -> **Stateless** (default choice). Poori app me ek shared object (counter/cache) -> **Singleton**.

7 Servlet Session vs JSP Session

Kya dono alag hain? Poori truth

7.1 Sabse pehle sach

EXAM / INTERVIEW POINT

Servlet session aur JSP session ek hi cheez hain - dono `javax.servlet.http.HttpSession` ka object. JSP internally servlet me convert hoti hai, aur us servlet ke `_jspService()` me container `session` naam ka variable (implicit object) apne aap bana deta hai. Isliye servlet me set kiya data JSP me milta hai aur JSP me set kiya data servlet me milta hai.

Java

```
// JSP ke translated servlet ke _jspService() ke andar (container likhta hai):
HttpSession session = null;
...
session = pageContext.getSession();     // <-- yahi hai JSP ka "session" implicit object
```

7.2 Difference Table

Point	Servlet me Session	JSP me Session
Object type	HttpSession	HttpSession (same)
Kaise milta hai	Khud mangna padta hai: `request.getSession()`	**Implicit object** hai: seedha `session` likho
Creation	Explicit: `getSession(true)` naya banata; `getSession(false)` nahi banata (null)	Page directive ka `session="true"` (default) hone par automatic bana/mil jata hai
Default behaviour	Jab tak getSession() call na karo, session nahi banta	Har JSP visit par session ban sakta hai (default session=true)
Disable karna	getSession() call hi mat karo / `getSession(false)`	`<%@ page session="false" %>`; phir `session` variable available nahi (compile error)
Null check	Zaroori (`getSession(false)` null de sakta hai)	Normally null nahi hota (session=true me)
Code style	Java class me	Scriptlet/EL me: `${sessionScope.user}`
Typical use	Login/logic: session set, validate, invalidate	Session ka data **display** karna
Scope	Session scope	Session scope (same)
Data sharing	Servlet me set -> JSP me dikhta hai	JSP me set -> Servlet me dikhta hai

7.3 Live demo: Servlet set kare, JSP read kare

Java

```
// LoginServlet.java
HttpSession session = request.getSession();            // servlet me explicit
session.setAttribute("user", "Ravi");
response.sendRedirect("welcome.jsp");
```

JSP

```
<%-- welcome.jsp  (session implicit object) --%>
<%
    String u = (String) session.getAttribute("user");  // same HttpSession object!
    out.println("Hello " + u);                          // Hello Ravi
%>
<%-- EL se --%>
Hello ${sessionScope.user}
```

WARNING

Hidden cost: JSP me default `session="true"` hai, isliye har JSP page open hone par server session bana sakta hai. Public pages (home, about, static info) par `<%@ page session="false" %>` lagao taaki memory bachi rahe.

8 JSP vs Servlet

Poora difference

8.1 Ek line me

Servlet = HTML ko Java code ke andar likhna. **JSP** = Java ko HTML ke andar likhna. Dono ek hi kaam karte hain (dynamic web content), aur JSP aakhir me ek Servlet ban jati hai.

8.2 13-point Difference

#	Point	Servlet	JSP
1	Definition	Java class jo request handle karti hai	Text-based page jisme HTML + Java tags
2	Code mix	HTML **Java ke andar** (`out.println("<h1>")`)	Java **HTML ke andar** (`<%= %>`)
3	Best for	Controller / business logic	View / presentation
4	MVC role	Controller	View
5	Ease	Mushkil (HTML strings, quotes)	Aasan; designer bhi kaam kar sakta hai
6	Compile	Developer compile karta hai (.class)	Container khud translate + compile karta hai
7	Change karne par	Recompile + redeploy/reload	File save karo, agli request me auto update
8	Extension / file	`.java` / `.class`	`.jsp`
9	Implicit objects	Nahi; khud `request.getSession()`, `getServletContext()` banao	9 implicit objects ready
10	Tag libraries / EL	Support nahi	JSTL, custom tags, EL support
11	Life cycle	init(), service(), destroy()	jspInit(), _jspService(), jspDestroy() (+ translation, compile)
12	Mapping	`@WebServlet` / web.xml me mapping	Direct URL se (`/page.jsp`), mapping zaroori nahi
13	Extends / implements	`HttpServlet` extend	Container ki generated class `HttpJspBase` extend karti hai

8.3 Same output, dono tareeke

Java

```
// Servlet version
public class HelloServlet extends HttpServlet {
    protected void doGet(HttpServletRequest req, HttpServletResponse res) throws IOException {
        res.setContentType("text/html");
        PrintWriter out = res.getWriter();
        out.println("<html><body><h1>Hello " + req.getParameter("name") + "</h1>");
        out.println("<p>Time: " + new java.util.Date() + "</p>");
        out.println("</body></html>");
    }
}
```

JSP

```
<%-- JSP version: same output, bahut chhota --%>
<html><body>
   <h1>Hello <%= request.getParameter("name") %></h1>
   <p>Time: <%= new java.util.Date() %></p>
</body></html>
```

EXAM / INTERVIEW POINT

Final answer: Servlet logic/control ke liye, JSP presentation ke liye. Real projects me dono saath: **Servlet (Controller) -> JSP (View)**.

9 Servlet & JSP Life Cycle

init - service - destroy | jspInit - _jspService - jspDestroy

9.1 Servlet Life Cycle

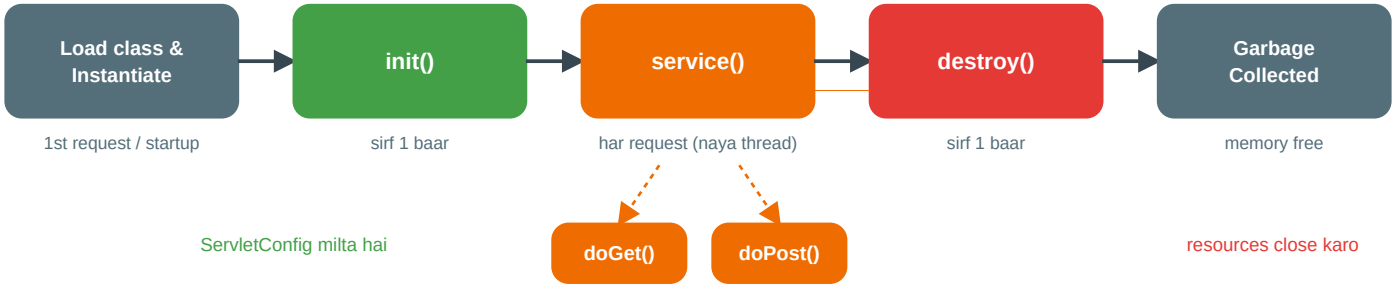


Fig 9.1 - Servlet lifecycle: container poori tarah control karta hai

Phase	Kaun karta hai	Kya hota hai	Kitni baar
Loading + Instantiation	Container	Class load hoti hai, default constructor se **sirf ek object** banta hai (1st request par, ya startup par agar `load-on-startup`)	1
`init(ServletConfig)`	Container	Initialization: DB connection, config padhna. Complete hone ke baad hi requests lete hain	1
`service(req,res)`	Container	Har request par **naya thread** isi object par chalta hai. HttpServlet ka service() method (GET/POST) dekhkar `doGet()/doPost()` call karta hai	Har request
`destroy()`	Container	Server band hone / app undeploy par. Resources close, data save	1

Java

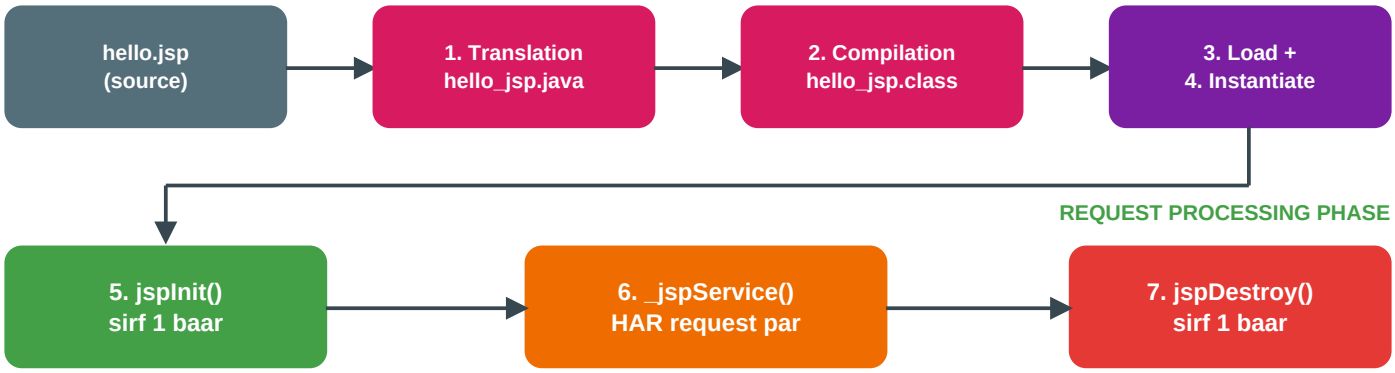
```
public class LifeServlet extends HttpServlet {
    public void init(ServletConfig cfg) throws ServletException {
        super.init(cfg);
        System.out.println("init()    - sirf ek baar");
    }
    protected void doGet(HttpServletRequest req, HttpServletResponse res) throws IOException {
        System.out.println("service -> doGet() - har request par");
        res.getWriter().println("Hello");
    }
    public void destroy() {
        System.out.println("destroy() - sirf ek baar");
    }
}
```

WARNING

Thread-safety: Servlet ka **sirf ek object** hota hai aur kai threads use share karte hain. Isliye servlet ke **instance variables me user-specific data mat rakho** (race condition). Local variables safe hain.

9.2 JSP Life Cycle

TRANSLATION PHASE (pehli request par ya JSP badalne par)



Mnemonic: T - C - L - I - I - R - D

(Translate, Compile, Load, Instantiate, Init, Request, Destroy)

Fig 9.2 - JSP lifecycle: 7 phases (Translation se Destroy tak)

Phase	Detail
1. Translation	JSP file ko **servlet source (hello_jsp.java)** me badla jata hai. JSP syntax error yahin pakda jata hai.
2. Compilation	hello_jsp.java -> **hello_jsp.class**. Java syntax error yahin aata hai.
3. Class loading	Class JVM me load hoti hai.
4. Instantiation	Generated servlet ka **ek instance** banta hai.
5. Initialization - `jspInit()`	Sirf **ek baar**. Resources/connection yahan.
6. Request processing - `_jspService()`	**Har request** par. Saara HTML + scriptlet yahin chalta hai.
7. Destroy - `jspDestroy()`	Ek baar, jab JSP hataayi jaaye / server band ho. Cleanup.

Teen JSP life-cycle methods ki detail

Method	Override kar sakte ho?	Kab	Kaam
`public void jspInit()`	Haan (declaration <%! %> me)	Ek baar, pehle request se pehle	Initialization: DB connection, config load
`public void _jspService(HttpServletRequest, HttpServletResponse)`	**Nahi** - container khud generate karta hai (underscore isi liye)	Har request	JSP ka poora body (HTML + scriptlet) isi method me jaata hai
`public void jspDestroy()`	Haan	Ek baar, destroy se pehle	Cleanup: connection close, file close

JSP

```
<%@ page import="java.sql.*" %>
<%!
    Connection con;                                 // instance variable (shared!)

    public void jspInit() {                         // 1 baar
        System.out.println("jspInit() called");
        // con = DriverManager.getConnection(...);
    }
    public void jspDestroy() {                      // 1 baar
        System.out.println("jspDestroy() called");
        // con.close();
    }
%>
<%
    out.println("_jspService() chal raha hai - har request par");   // _jspService ka hissa
%>
```

Container jo servlet generate karta hai (simplified)

Java

```
public final class hello_jsp extends HttpJspBase {
    int visits = 0;                                  // <%! %> declaration yahan aata hai
    public void jspInit()    { ... }                 // aapka override
    public void jspDestroy() { ... }

    public void _jspService(HttpServletRequest request, HttpServletResponse response)
            throws IOException, ServletException {
        HttpSession session = null;  ServletContext application = null;     // implicit
        ServletConfig config = null; JspWriter out = null; Object page = this;
        PageContext pageContext = null;
        ...
        out.write("<html><body>");                   // template text
        out.print( request.getParameter("name") );   // <%= %> expression
        out.write("</body></html>");
    }
}
```

9.3 Servlet vs JSP lifecycle - ek nazar me

Servlet	JSP	Kitni baar
(developer compile karta hai)	Translation + Compilation (container)	Pehli request / change par
`init()`	`jspInit()`	1 baar
`service() -> doGet/doPost`	`_jspService()`	Har request
`destroy()`	`jspDestroy()`	1 baar

EXAM / INTERVIEW POINT

Yaad rakhne ka trick: JSP lifecycle = **T-C-L-I-I-R-D** (Translate, Compile, Load, Instantiate, Init, Request, Destroy). Aur `_jspService()` ko tum override nahi kar sakte kyunki wo JSP ke body se generate hota hai.

10 What is JSP & Why JSP

JavaServer Pages ka pura concept

10.1 JSP kya hai?

JSP (JavaServer Pages) ek **server-side technology** hai jisse hum **HTML ke andar Java code** embed karke **dynamic web pages** banate hain. File ka extension `.jsp` hota hai. Container is page ko **Servlet me translate** karke chalata hai, isliye JSP asal me "servlet ka aasan roop" hai.

10.2 Servlet ke problems -> JSP ka janm

- Servlet me HTML `out.println("...")` se likhna padta hai - lamba, quotes ki jhanjhat, padhna mushkil.
- Web designer Java nahi jaanta, wo servlet ko edit nahi kar sakta.
- Chhota sa text/colour badalne par bhi recompile + redeploy.
- Presentation aur logic mix ho jate hain.

10.3 Why JSP - Fayde

Fayda	Explanation
Easy to write	HTML jaisa likho, jahan dynamic data chahiye wahan <%= %> / EL.
Separation of roles	Designer HTML part, developer Java/beans.
Auto compile	Change karo, save karo, agli request me naya version (redeploy nahi).
Implicit objects	request, session, out... bina declare kiye.
Tag libraries	JSTL, custom tags se scriptlet kam, code clean.
Full Java power	JDBC, collections, beans, EJB sab use kar sakte ho.
Platform independent	Write once, run anywhere (JVM).
Fast at runtime	Compile hone ke baad servlet jaisi speed.
Part of Java EE	Servlet, JSTL, JSF, EL ke saath integrate.

10.4 JSP page ki anatomy

JSP

```
<%@ page language="java" contentType="text/html; charset=UTF-8" %>   <!-- 1. Directive -->
<%@ taglib uri="http://java.sun.com/jsp/jstl/core" prefix="c" %>      <!-- 1. Directive -->
<%! int count = 0; %>                                                 <!-- 2. Declaration -->
<html>
<body>                                                                <!-- 3. Template text (HTML) -->
   <% count++; String user = request.getParameter("u"); %>            <!-- 4. Scriptlet -->
   <h1>Welcome <%= user %></h1>                                       <!-- 5. Expression -->
   <p>Visit No: ${count}</p>                                          <!-- 6. EL -->
   <c:if test="${not empty param.u}">Namaste!</c:if>                  <!-- 7. JSTL -->
   <jsp:include page="footer.jsp" />                                  <!-- 8. Action tag -->
</body>
</html>
```

> **NOTE**
> **JSP = Template text (HTML) + JSP elements.** Template text jaisa hai waisa response me chala jata hai; JSP elements server pe execute hote hain.

10.5 JSP ki limitations

- Debugging servlet se mushkil (line numbers translated file ke aate hain).
- Zyada scriptlet likhne par code spaghetti ban jata hai - isliye MVC + JSTL/EL use karo.
- Pehli request par translation + compilation ka thoda delay.

- Heavy business logic JSP me nahi likhni chahiye.

> **TIP**
> **Modern practice:** JSP me scriptlet (<% %>) avoid karo; EL (${}) + JSTL use karo, logic Servlet/Service me rakho. Aajkal Thymeleaf/JSF/React bhi view ke liye use hote hain, par JSP syllabus aur legacy projects me abhi bhi bahut hai.

11 Which is Faster: JSP or Servlet?

Seedha jawab aur reason

11.1 Answer

EXAM / INTERVIEW POINT

Runtime me dono lagbhag same speed (kyunki JSP aakhir me servlet hi ban jati hai aur wahi .class chalti hai). **Pehli request par JSP slow** hoti hai (translation + compilation). **Servlet thoda sa fast** maana jata hai kyunki ye extra step nahi hota. **Development me JSP fast** hai.

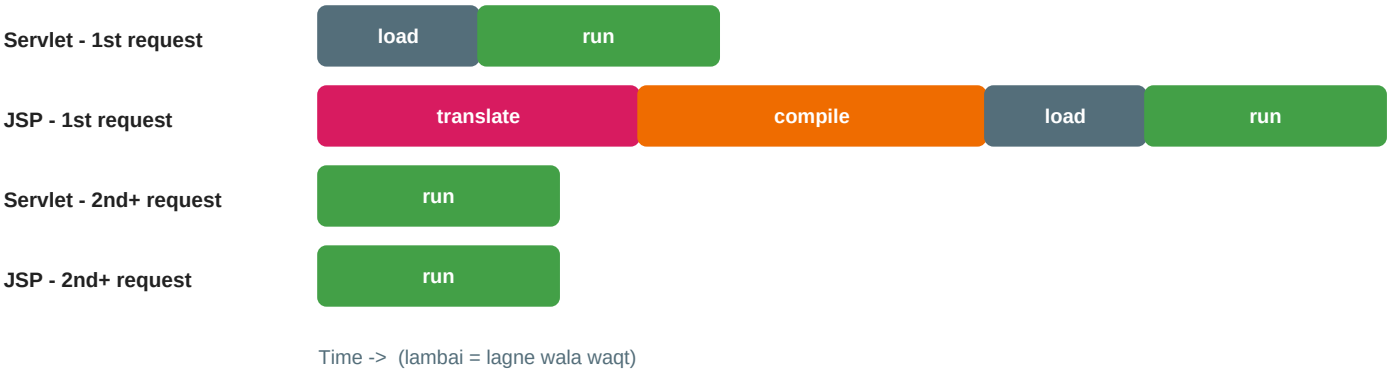


Fig 11.1 - Pehli request par JSP ko extra translate + compile karna padta hai; baad me dono barabar

11.2 Comparison

Factor	Servlet	JSP
First request	Class load + init	**Translate + compile + load + init** (slow)
2nd request onwards	Fast	Fast (same as servlet)
Generated code	Jo aapne likha	Container ka generated code (thoda extra out.write calls)
Development speed	Slow (HTML strings)	**Fast**
Maintenance	Presentation badalna mushkil	Easy
Overall winner	Runtime micro-advantage	Productivity advantage

11.3 JSP ko fast banane ke tips

- **Precompile** JSPs (Tomcat ka `jspc` / build time) taaki pehli request slow na ho.
- Jahan session nahi chahiye wahan `<%@ page session="false" %>`.
- Static content ke liye `<%@ include %>` (compile time) use karo, heavy loops/logic servlet/service me.
- Connection **pooling** (DataSource), `PreparedStatement`, caching.
- Output **buffer** theek rakho (`buffer="16kb"`), gzip compression, bade objects session me mat daalo.

TIP

Interview line: "Speed ka difference negligible hai. JSP ek baar compile hone ke baad servlet ki tarah chalti hai. Choose karna ho to logic ke liye servlet, view ke liye JSP."

12 9 Implicit Objects of JSP

Container ke diye hue ready objects

12.1 Implicit objects kya hain?

JSP ke andar kuch objects **bina declare/create kiye** use kar sakte hain. Container inhe generated servlet ke `_jspService()` me **apne aap bana deta hai**. Total **9** hain.

> **WARNING**
> Implicit objects sirf **scriptlet** <% %> **aur expression** <%= %> me milte hain, **declaration** <%! %> **me nahi** (kyunki wo _jspService ke bahar class level par jaata hai).

#	Object	Type (class)	Scope	Servlet me equivalent
1	**request**	HttpServletRequest	request	doGet/doPost ka req
2	**response**	HttpServletResponse	page	doGet/doPost ka res
3	**out**	JspWriter	page	res.getWriter()
4	**session**	HttpSession	session	req.getSession()
5	**application**	ServletContext	application	getServletContext()
6	**config**	ServletConfig	page	getServletConfig()
7	**pageContext**	PageContext	page	(JSP-specific)
8	**page**	Object (this)	page	this
9	**exception**	Throwable	page	(error page only)

12.2 Scope ka diagram

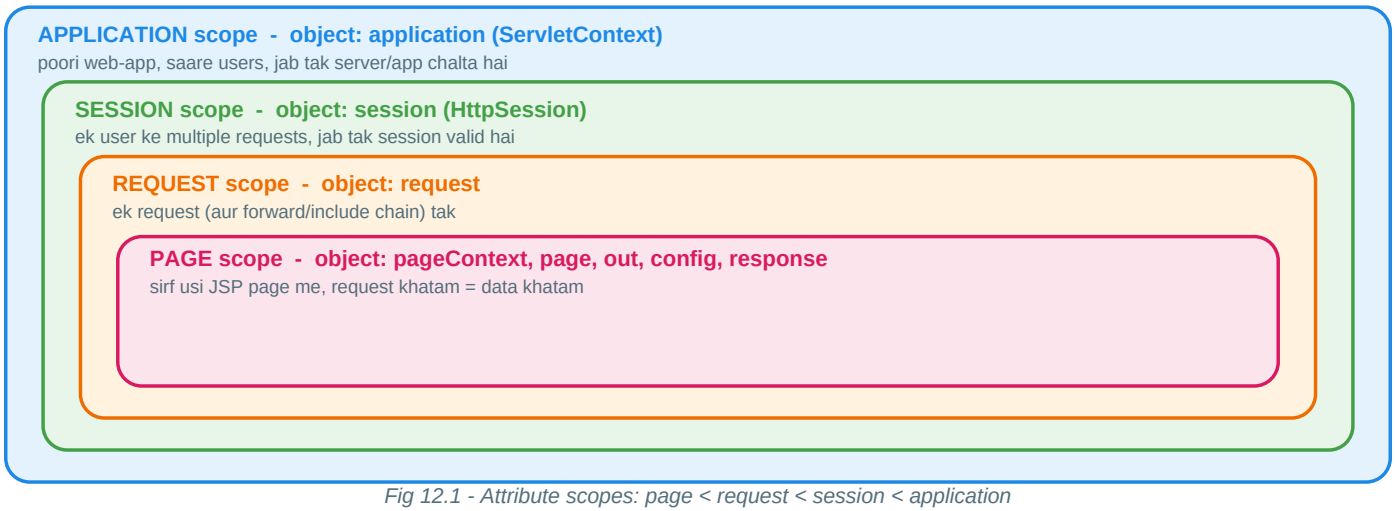


Fig 12.1 - Attribute scopes: page < request < session < application

12.3 Har object detail me

1. request — `HttpServletRequest`

Client ki request ka data: form parameters, headers, cookies, IP. **Request scope** me attribute bhi rakh sakte ho.

```
String user = request.getParameter("user");

request.setAttribute("msg", "Hi");

String ip = request.getRemoteAddr();
```

2. response — `HttpServletResponse`

Client ko jaane wala response control karna: redirect, content type, cookie, header.

```
response.sendRedirect("home.jsp");

response.setContentType("text/html");

response.addCookie(new Cookie("lang", "hi"));
```

3. out — `JspWriter`

Response body me text likhne ka buffered writer (PrintWriter jaisa, par buffer ke saath).

```
out.println("Hello " + name);

out.flush();   // buffer ko bhej do
```

4. session — `HttpSession`

Ek user ke multiple requests ke beech data share karta hai (login, cart).

```
session.setAttribute("user", "Rahul");

String u = (String) session.getAttribute("user");

session.invalidate();
```

5. application — `ServletContext`

Poori web-app ka shared object: sab users, sab pages. Global counter, context-param, real path.

```
application.setAttribute("hits", 10);

String db = application.getInitParameter("dbUrl");   // web.xml context-param

String path = application.getRealPath("/");
```

6. config — `ServletConfig`

Is JSP/servlet ka configuration (web.xml ke init-param, servlet name).

```
String theme = config.getInitParameter("theme");

String nm = config.getServletName();
```

7. pageContext — `PageContext`

Sabka boss: baaki **saare implicit objects** tak pahunch deta hai + **kisi bhi scope** me attribute set/get.

```
pageContext.setAttribute("x", "1", PageContext.SESSION_SCOPE);

Object v = pageContext.findAttribute("x");   // page->request->session->app

pageContext.forward("a.jsp");   pageContext.getRequest();
```

8. page — `java.lang.Object`

Current JSP ka generated servlet instance (**this** ka reference). Rarely use hota hai.

```
Object p = page;

out.println(page.getClass().getName());
```

9. exception — `java.lang.Throwable`

Sirf **error page** me milta hai (`isErrorPage="true"`). Pichli page ki exception ka object.

```
<%@ page isErrorPage="true" %>

<%= exception.getMessage() %>

<%-- source page: <%@ page errorPage="error.jsp" %> --%>
```

EXAM / INTERVIEW POINT

Yaad karne ka trick: R R O S A C P P E = **R**equest, **R**esponse, **O**ut, **S**ession, **A**pplication, **C**onfig, **P**ageContext, **P**age, **E**xception. **Exception** ko chhodkar baaki 8 har JSP me available hain.

13 JSP Elements & Scriptlet

JSP me kya-kya likh sakte hain

13.1 JSP elements ki family

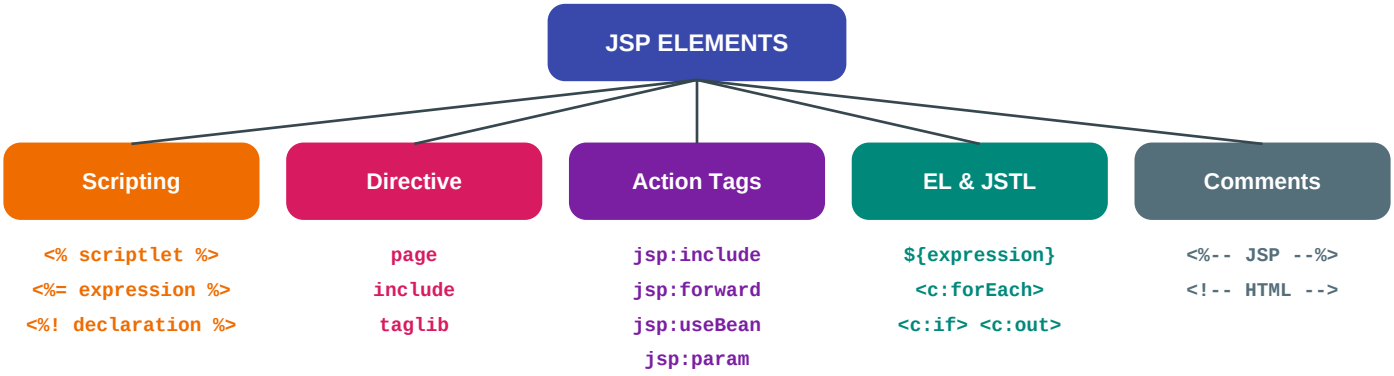


Fig 13.1 - JSP ke saare elements

Element	Syntax	Translated servlet me kahan jaata hai	Kaam
Scriptlet	`<% code; %>`	`_jspService()` ke **andar** (local)	Java statements, loops, conditions
Expression	`<%= expr %>`	`out.print(expr);`	Value output me print karna (semicolon nahi)
Declaration	`<%! decl %>`	Class level (**_jspService ke bahar**)	Variables aur methods declare karna
Directive	`<%@ ... %>`	Translation time par container ko instruction	page, include, taglib
Action tag	`<jsp:xxx />`	Runtime par kaam (method calls)	include, forward, useBean...
Comment	`<%-- --%>`	Translate hi nahi hota	Hidden comment
EL / JSTL	`${ }`, `<c:xxx>`	Runtime evaluation	Scriptlet-free clean code

13.2 Scriptlet <% ... %>

Scriptlet me **koi bhi valid Java statements** likh sakte ho. Ye code `_jspService()` ke andar jaata hai, isliye isme declare kiye variable **local** hote hain (har request me naye). Scriptlet me **method declare nahi kar sakte** (method ke andar method nahi hota).

JSP

```
<%
    int marks = Integer.parseInt(request.getParameter("m"));
    String grade;
    if (marks >= 75) grade = "A"; else grade = "B";
%>
<p>Grade: <%= grade %></p>

<% for (int i = 1; i <= 3; i++) { %>       <%-- HTML ko loop ke beech me tod sakte ho --%>
    <li>Item <%= i %></li>
<% } %>
```

13.3 Expression <%= ... %>

- Result ko **String me convert** karke seedha output me likhta hai: `out.print(expression)`.
- **Semicolon (;) nahi lagana** - `<%= x; %>` galat hai (`out.print(x;);` ban jayega).
- Ek expression me ek hi value: `<%= a + b %>`, `<%= new java.util.Date() %>`, `<%= user.toUpperCase() %>`.

13.4 Declaration <%! ... %>

Variables aur **methods** class level par declare karta hai. Ye generated class ke **instance variable/method** ban jaate hain, isliye **sab requests (threads) ke beech shared** hote hain.

JSP

```
<%!
    int visits = 0;                                  // instance variable - SHARED by all users
    String greet(String n) { return "Namaste " + n; }  // method
    public void jspInit() { /* 1 baar */ }           // lifecycle method override
%>
```

> **WARNING**
> **Thread-safety:** `<%! int visits = 0; %>` sab users ke liye ek hi variable hai - ek saath kai users se race condition ho sakti hai. Isliye per-request data scriptlet ke **local** variable me rakho. Aur `<%! %>` me implicit objects (request, session) nahi milte.

Scriptlet vs Expression vs Declaration

Point	Scriptlet	Expression	Declaration
Tag	<% %>	<%= %>	<%! %>
Goes to	_jspService() ke andar	out.print() ke andar	Class level
Variable type	Local	-	Instance (shared)
Methods	Nahi bana sakte	Nahi	**Haan**
Semicolon	Zaroori	**Nahi**	Zaroori
Implicit objects	Milte hain	Milte hain	**Nahi milte**

13.5 Comments

Type	Syntax	Client ko dikhta hai?
JSP comment	`<%-- note --%>`	**Nahi** (translate hi nahi hota)
HTML comment	`<!-- note -->`	**Haan** (view-source me dikhta hai)
Java comment	`<% // note /* note */ %>`	Nahi (servlet source me rehta hai)

13.6 Directives <%@ ... %>

Directive **translation time** par container ko instruction deta hai ki page kaise translate karna hai. Teen types:

(a) page directive

Attribute	Kaam	Example
import	Java packages import	`import="java.util.*,java.sql.*"`
contentType	Response MIME + charset	`contentType="text/html;charset=UTF-8"`
session	Session enable/disable (default true)	`session="false"`
errorPage / isErrorPage	Exception par kis page par jana / ye error page hai	`errorPage="error.jsp"`
buffer / autoFlush	Output buffer size / auto flush	`buffer="16kb"`
isELIgnored	EL ignore karna	`isELIgnored="false"`
language / extends / pageEncoding / isThreadSafe	Language (java), parent class, source encoding, thread model	`pageEncoding="UTF-8"`

(b) include directive

JSP

```
<%@ include file="header.jsp" %>      <%-- translation time par content copy ho jaata hai --%>
```

(c) taglib directive

```jsp
<%@ taglib uri="http://java.sun.com/jsp/jstl/core" prefix="c" %>      <%-- JSTL tags ke liye --%>
```

13.7 Action tags

Tag	Kaam
`<jsp:include page="a.jsp" />`	Runtime par dusre resource ka **output** include
`<jsp:forward page="b.jsp" />`	Control dusre resource ko de do (URL same rehta hai)
`<jsp:param name="k" value="v" />`	include/forward ke saath parameter bhejna
`<jsp:useBean> / setProperty / getProperty`	JavaBean banana/bharna/padhna (Chapter 3)
`<jsp:plugin>`	Applet/plugin embed (old)

include directive vs include action

Point	<%@ include file="" %>	
Kab	**Translation time**	**Request (run) time**
Kya include	Source code copy	Output (response) include
Result	Ek hi servlet banta hai	Alag servlet, runtime par call
Speed	Thoda fast	Thoda slow, par flexible
Included file badle	Parent ko dobara compile karna padega	Seedha reflect hota hai
Use	Static: header/footer	Dynamic content

13.8 Complete example: sab elements saath me

```jsp
<%@ page import="java.util.*" contentType="text/html;charset=UTF-8" %>
<%!
    int visits = 0;
    String greet(String n) { return "Namaste " + n; }
%>
<html><body>
<%
    visits++;
    String name = request.getParameter("n");
    if (name == null) name = "Guest";
%>
<h2><%= greet(name) %></h2>
<p>Is page ko <%= visits %> baar dekha gaya.</p>
<ul>
<% for (int i = 1; i <= 3; i++) { %>
    <li>Item <%= i %></li>
<% } %>
</ul>
</body></html>
```

Translate hone par (simplified)

```java
public class demo_jsp extends HttpJspBase {
    int visits = 0;                                          // <%! %>  -> class level
    String greet(String n) { return "Namaste " + n; }        // <%! %>  -> class level

    public void _jspService(HttpServletRequest request, HttpServletResponse response) ... {
        visits++;                                            // <% %>   -> method ke andar
        String name = request.getParameter("n");
        if (name == null) name = "Guest";
        out.print( greet(name) );                            // <%= %>  -> out.print()
        out.print( visits );
        for (int i = 1; i <= 3; i++) { out.write("<li>Item "); out.print(i); out.write("</li>"); }
    }
}
```

TIP

Best practice: Scriptlet kam, **EL + JSTL** zyada. Upar ka loop JSTL me: `<c:forEach begin="1" end="3" var="i"><li>Item ${i}</li></c:forEach>`

14 Revision, Q&A, Cheat Sheet

Exam se pehle yahi padh lo

14.1 One-page Cheat Sheet

Topic	Yaad rakhne wali line
MVC	Model = data+logic (Bean/DAO), View = JSP, Controller = Servlet. Request -> Controller -> Model -> Controller -> View -> Response.
Architecture	Client -> Web tier (Servlet/JSP, Tomcat) -> Business tier (EJB) -> Data tier (DB). Tomcat = web container only.
JavaBean	public class + no-arg constructor + private fields + getters/setters + Serializable.
EJB	Server-side business component in EJB container: transaction, security, pooling free. Types: Session, MDB (Entity -> JPA).
Session	HTTP stateless; HttpSession + JSESSIONID cookie; `getSession(false)` null de sakta hai; `invalidate()` logout.
Session Bean	Stateless (pool, no state), Stateful (1 client = 1 bean), Singleton (1 per app). Not same as HttpSession.
Servlet vs JSP session	Same HttpSession object. JSP me implicit `session`, default session="true".
Servlet lifecycle	init() once -> service() every request -> destroy() once. Ek instance, multi-thread.
JSP lifecycle	Translate -> Compile -> Load -> Instantiate -> jspInit() -> _jspService() -> jspDestroy().
JSP vs Servlet	Servlet = Java me HTML (controller). JSP = HTML me Java (view). JSP finally servlet banti hai.
Speed	Runtime same; JSP first request slow (translate+compile).
9 objects	request, response, out, session, application, config, pageContext, page, exception.
Elements	<% %> scriptlet, <%= %> expression, <%! %> declaration, <%@ %> directive, <jsp:*> action.

14.2 Interview / Exam Questions (short answers)

Q1. MVC kyon use karte hain?

Separation of concerns: UI, logic aur control alag hone se maintenance, reuse aur testing aasan hoti hai.

Q2. Servlet me MVC ke 3 parts kya hain?

Model = JavaBean/DAO, View = JSP, Controller = Servlet.

Q3. JavaBean ke rules?

Public class, public no-arg constructor, private properties, public getters/setters, Serializable.

Q4. EJB ka container kya deta hai?

Transaction, security, pooling, concurrency, lifecycle, DI, remote access.

Q5. Stateless aur Stateful bean me fark?

Stateless client ki state nahi rakhta aur pool se shared hota hai; Stateful har client ke liye alag instance aur state yaad rakhta hai.

Q6. HttpSession aur Session Bean same hain?

Nahi. HttpSession web tier ka user-data object hai; Session Bean EJB tier ka business component hai.

Q7. getSession() aur getSession(false) ka fark?

getSession() session na ho to naya banata hai; getSession(false) null return karta hai.

Q8. Servlet aur JSP session alag hain?

Nahi, dono HttpSession hain. JSP me wo implicit object session ke naam se milta hai.

Q9. JSP ka life cycle bolo.

Translation, compilation, loading, instantiation, jspInit(), _jspService(), jspDestroy().

Q10. _jspService() override kyun nahi hota?

Kyunki container ise JSP ke body se generate karta hai; naam me underscore isi ke liye.

Q11. Servlet ke instance variables unsafe kyun?

Servlet ka ek hi object hai aur kai threads share karte hain, to race condition ho sakti hai.

Q12. include directive vs action?
Directive translation time par source copy karta hai; action runtime par output include karta hai.

Q13. Expression me semicolon kyun nahi?
Kyunki wo out.print(expr) ke andar jaata hai; semicolon lagane se syntax error.

Q14. JSP me kaun sa object exception handling ke liye?
exception, jo sirf isErrorPage="true" wale page me milta hai.

Q15. forward aur sendRedirect ka fark?
forward server-side hai (URL same, same request, fast); sendRedirect client-side hai (nayi request, URL badalta hai).

14.3 Common galtiyan

- JSP me DB code/logic bhar dena (MVC todna).
- Servlet me instance variable me user data rakhna.
- `<%= x; %>` me semicolon lagana.
- `<%! %>` me `request` ya `session` use karna.
- EJB ko `new` se banana.
- `getSession(false)` ke result ko null check kiye bina use karna.
- Logout par `session.invalidate()` bhoolna.

> **TIP**
> **All the best!** Sabse pehle MVC aur lifecycle diagrams ko khud draw karke practice karo - exam me diagram + 2 line explanation full marks dilata hai.